A Versatile Cybersecurity Development Lifecycle (*AVCDL*)

Revision

Version 64
4/6/26 6:04 PM

Author

Charles Wilson

License

Table of Contents

List of Tables

List of Figures

1. Introduction

1.1 Abstract

This material documents A Versatile Cybersecurity Development Lifecycle (***AVCDL***).

1.2 Organization

This document is organized as follows:

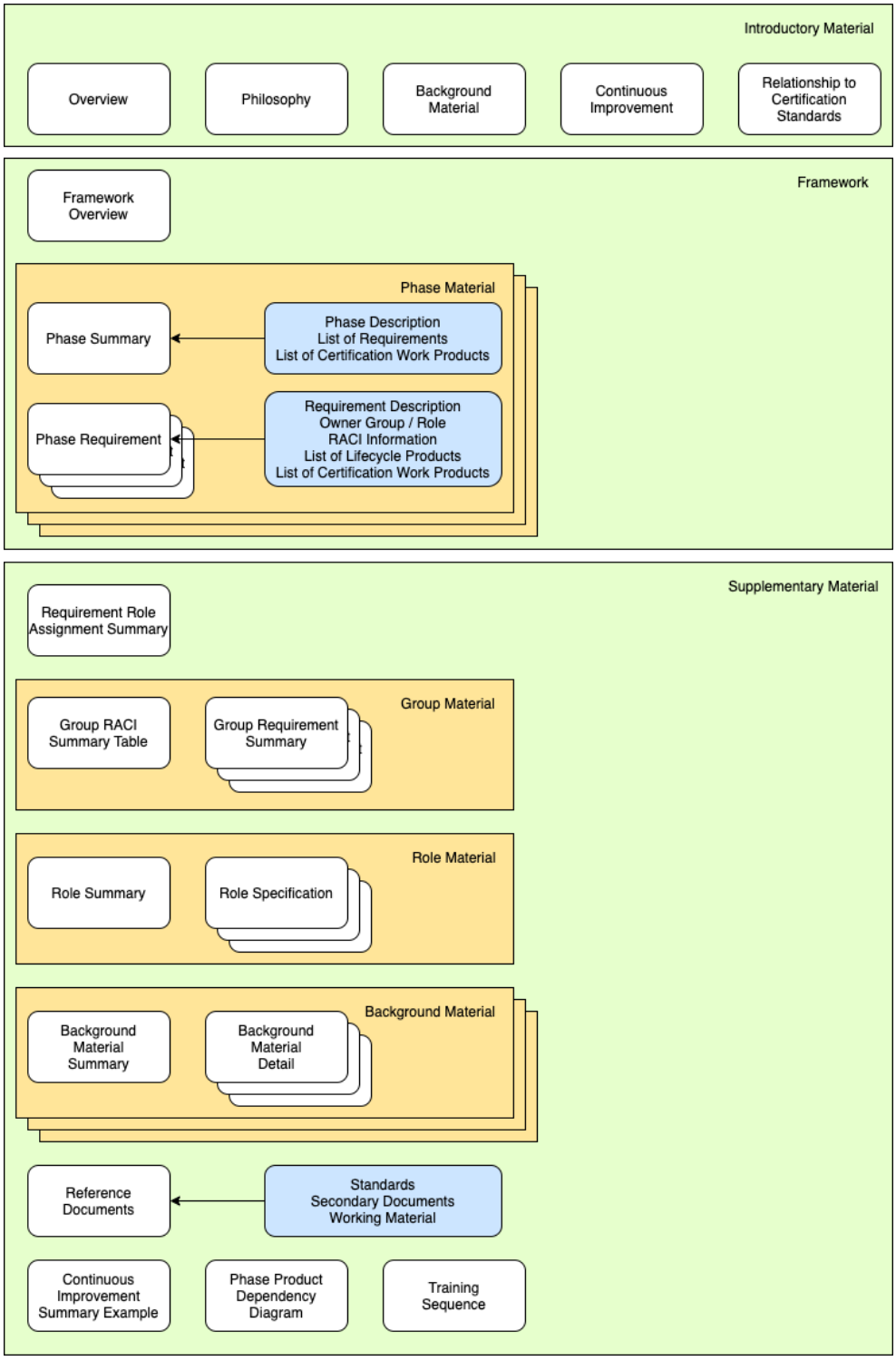

Figure 1 - Document Organization

As can be seen, there are three sections:

- Introductory material
- Framework (main AVCDL material)
- Supplementary material

1.2.1 Introductory Material

The introductory material provides a general overview of the **AVCDL**, its philosophy, and background material. It also discusses how continuous improvement is measured and tracked. The relationship between the **AVCDL** and **ISO/SAE 21434** is explored, as is how hardware and software are treated.

1.2.2 Framework

This is the core of the **AVCDL**. An overview of the AVCDL framework is presented. Following that each phase of the lifecycle is summarized, and the associated phase requirements are detailed.

1.2.3 Supplementary Material

The supplementary material falls into two broad categories. The first is background material. These include **NCWF** [10], **MSSDL**, and **SSDF**. The second is summary material. This includes RACI rollup, secondary document lists, a continuous improvement example, phase dependency graphs and recommended training sequence.

Note: Both the main **AVCDL** and its secondary documents are process in nature. They do not specify the tools and specific techniques used to accomplish their tasks. As such, the **AVCDL** is suitable for use by any organization engaged in development related to autonomous vehicles. It is presumed that each organization adopting the **AVCDL** will create a set of tertiary documents supporting the secondary ones. These tertiary documents will be procedural. Additionally, there may be multiple tertiary documents supporting a given secondary document. For example, a secondary document on secure code review might be supported by multiple tertiary documents, one for each programming language.

2. Overview

2.1 What it is

The **AVCDL** is a set of identified processes, requirements of those processes, generated products for the purpose of ensuring the creation of secure safety-critical, cyber-physical systems. It is intended to support auditing of the development process in cybersecurity as specified in those standards. It is intended to comply with cybersecurity standards and regulations such as **ISO/SAE 21434**, **ISO 24089**, **UN R155**, and **UN R156**.

2.2 What it isn't

The **AVCDL** does not attempt to specify:

- implementation methodology (waterfall, V-model, agile-scrum, agile-Kanban, TDD, BDD, spiral, …)
- specific development tools (source code control, build system, compilers, threat modeling tools, static analysis tools, …)
- remediation methodology

2.3 Where it fits

The **AVCDL** is not a standalone solution. It is intended to implement the non-governance elements of a larger product development lifecycle (**AVPDL**). Moreover, it is designed to overlay the system (**ISO/IEC 15288**) and software (**ISO/IEC 12207**) lifecycles; and complement a safety lifecycle such as **ISO 26262**.

We can visualize the relationship between the **AVCDL,** and various standards as follows:

AVPDL	AVCDL	15288	12207	26262	21434
organization processes	N/A	technical processes	technical processes	management of functional safety	overall cybersecurity management
				supporting processes	project dependent cybersecurity management
foundation phase	*foundation phase*	N/A	N/A	concept phase	concept phase
requirements phase	*requirements phase*	requirements definition	requirements definition	safety requirements	cybersecurity requirements
		requirements analysis	system requirements analysis	hazard analysis / risk assessment	cybersecurity assessment
design phase	*design phase*	architectural design	system architectural design	architectural design	cybersecurity design
implementation phase	*implementation phase*	implementation	implementation	implementation	development
		integration	system integration	integration and verification	integration and verification
verification phase	*verification phase*	verification	system qualification testing		
		transition	software installation		
			software acceptance support		
release phase	*release phase*	validation		production	production
operation phase	*operation phase*	operation	software operation	operation, service, and decommissioning	continuous cybersecurity activities
		maintenance	software maintenance		operation and maintenance
decommissioning phase	*decommissioning phase*	disposal	software disposal		decommissioning
supplier processes	N/A	agreement processes	agreement processes	supporting processes	distributed cybersecurity activities

Table 1 - Relationship Among Standards

Note: The **AVCDL** does not attempt to address either the organization or supplier-related processes managed at the organizational level.

3. Philosophy

The creation of secure software is not simply a programming endeavor. It begins with ensuring all team members understand how software and systems are made secure; requires the additional stages in the design and testing phase; and permeates all typical development practices.

It is impossible to integrate security into a large operational system in a single pass. This is a reality stemming from the lack of security focus within the computing industry. To be effective, security must be seen as emergent property and not an adjunct capability. This stands in contrast to the recent trend toward minimal functional development. For a system to be secure it must be secure by design, not coincidence. Therefore, there are some foundational elements which should be in place prior to implementation.

Given the scope of the problem space, the most appropriate approach entails:

- creation of an ***implementation framework***
- ***continuous improvement*** of the security posture

3.1 Organizational Participation

Beyond being a collection of activities, the ***AVCDL*** has implicit in its structure and implementation the participation of various groups throughout the organization. As can be seen from the previous section (***Where It Fits***), **ISO/SAE 21434** is just one standard among many that must be satisfied. Each of these standards is primarily the responsibility of a single group (**ISO/IEC/IEEE 15288** – systems engineering, **ISO/IEC/IEEE 12207** – software development, **ISO 26262** - safety). Beyond these are the project and product management teams, as well as the other organization groups, such as devops and quality management. To achieve compliance with these standards, it is necessary for there to be both communication and coordination of activities between these groups.

Although most of the activities described in the ***AVCDL*** are the responsibility of the cybersecurity group, they are not solely responsible. This distribution of responsibilities is summarized in the ***Groups*** section of the document. Additionally, the specific responsibilities of each activity are documented both in a summary form in each phase requirement and a highly detailed form in each of the ***AVCDL*** secondary documents which show activity workflows.

3.2 Information Sharing

Any sufficiently complex product will require contributions from multiple groups within the organization. This being the case, it is a base assumption of the ***AVCDL*** that information sharing between groups is established within an organization. All input materials feeding, and output materials generated in the execution of ***AVCDL*** phase requirement activities are presumed to be made available to the organization within the context of appropriate information sharing as dictated by the organization's communication policies. It is further presumed that mechanisms exist to inform various groups when the state of shared information becomes available or changes. It is outside the scope of the ***AVCDL*** as to either the underlying organizational policies, groups involved, or notification mechanisms used by any organization in implementing information sharing. The specific external information required is indicated in each phase requirement in the **External Group Dependencies** section.

3.3 Inter-group Communication

It is not the intent of the ***AVCDL*** to dictate or elaborate on the necessity of effective inter-group communication within an organization. It is assumed that where non-cybersecurity groups are involved in the fulfillment of phase requirements that the management of all involved groups ensure a timely and cooperative level of engagement. This is, of course, the responsibility of the organization's leadership, but bears repeating.

Each phase requirement clearly indicates the four main groups involved in their completion. Additionally, as mentioned above a summary of these primary groups and their level of responsibility is provided in the ***Groups*** section of the document. Additional elaboration on the individuals involved in the execution of the workflows used to attain the work products for the phase requirements is provided in each of the secondary documents associated with the various phase requirements. It is presumed that these will be used not only for project planning, but also to ensure that the appropriate groups are engaged in a timely manner.

3.4 Reference to Groups

As mentioned above, within the ***AVCDL*** document set there are four primary groups responsible for the execution of the phase requirements. These are:

3.4.1 Cybersecurity

This group is the primary focus of the ***AVCDL*** documents. Alternately, the term **security** may be used when referring to the group or its members.

3.4.2 Development

This is the group responsible for the creation of the product. It is presumed to include all teams responsible for creation and implementation of the product's functional requirements. These include systems, hardware, and software development functions.

3.4.3 DevOps

This is the group responsible for the creation and management of the software and systems used in the assembly of the product's software. It may also be responsible for the systems used to track hardware related metadata. This may exist either as a distinct functional group or be part of the development organization. Within the context of the ***AVCDL*** it will be treated as a distinct group.

3.4.4 Risk

This is the group responsible for the assessment of risk to the product. Members of this groups are presumed to have expertise in one of the four areas of risk identified in **EVITA D2.3 - Security requirements for automotive on-board networks based on dark-side scenarios** section 2.7 **Overview of risk analysis** [16]. The risk domains are safety, finance, operation, and privacy. These may exist either within a distinct functional group or be part of their respective groups within the organization. The distinction depends upon the size and maturity of the organization. Within the context of the ***AVCDL*** it will be treated as a distinct group capable of assessing risk from any of the abovementioned risk areas. When used generically, **risk** may be considered to refer to either the general or specific category, depending upon the audience.

4. Background Material

The ***AVCDL*** is based on methodologies proven in industry as well as standards bodies' recommendations.

4.1 Contribution Sources

The following are the major sources of contribution to the ***AVCDL***.

4.1.1 Microsoft SDL (MSSDL) [6]

The archetype for the cybersecurity development lifecycle is the Microsoft SDL (**MSSDL**). It divides the development process into seven phases. These phases form a cycle of ever-improving security posture.

4.1.2 NIST SSDF (SSDF) [11]

The NIST Secure Software Development Framework (**SSDF**) provides a more general approach which calls out several practices and provides references to the applicable standards. Within each of these are multiple practices and tasks.

The advantage of the **SSDF** over the **MSSDL** is that it provides a greater level of specificity and better supports existing international standards. It also calls out practices assumed but not specified in the **MSSDL**.

4.1.3 ISO/SAE 21434 [4]

Road Vehicles - Cybersecurity Engineering (**ISO/SAE 21434**) is intended to address the cybersecurity aspects of electrical and electronic (E/E) systems within road vehicles. Its goal is to enable organizations to:

- define cybersecurity policies and processes
- manage cybersecurity risk
- foster a cybersecurity culture

Like the **MSSDL**, the development process is divided into phases.

Unlike **MSSDL** and **SSDF**, which are lifecycle-focused, non-domain-specific documents, **ISO/SAE 21434** is a regulatory-focused, domain-specific (road vehicle E/E systems) work.

4.1.4 ISO 24089 [18]

Road Vehicles – Software Update Engineering (**ISO 24089**) is intended to address the software update processes of road vehicles. These include the consideration of cybersecurity.

4.1.5 ISO 26262 [17]

Road vehicles — Functional safety (**ISO 26262**) is intended to address the safety aspects of electrical and electronic (E/E) systems within road vehicles. Although not used as a primary source reference for the **AVCDL**, the **AVCDL** can be aligned to it. This allows for easier integration with existing development processes.

4.1.6 ISO/IEC/IEEE 15288 [3]

Systems and software engineering – System life cycle processes (**ISO/IEC/IEEE 15288**) provides a set of processes required to systematically implement a system development lifecycle. This should be considered the grounding process alignment document.

4.1.7 ISO/IEC/IEEE 12207 [2]

Systems and software engineering – Software life cycle processes (**ISO/IEC/IEEE 12207**) provides a set of processes required to systematically implement a software development lifecycle. This should be considered the grounding process alignment document.

4.1.8 UN Regulations

The United Nations' Economic Commission for Europe (**ECE**) Inland Transport Committee's World Forum for Harmonization of Vehicle Regulations' Working Party on Automated/Autonomous and Connected Vehicles (**WP29** [15]) has created a set of automotive cybersecurity regulations. Their intent is to

- [define] principles to address key cyber threats and vulnerabilities identified in order to assure vehicle safety in case of cyber-attacks
- [define] detailed guidance or measures for how to meet these principles … [including] examples of processes and technical approaches
- [consider] what assessments or evidence may be required to demonstrate compliance or certification with any requirements identified

These documents are intended to provide a common set of definitions and principles. For implementations, they point to specific standards, such as **ISO 26262**, **ISO/SAE 21434** and **ITU-T X.1500**. **UN** regulations **R155** [19] and **R156** [20] are the ones of specific interest.

4.2 Contributions Visualized

These sources come together as follows:

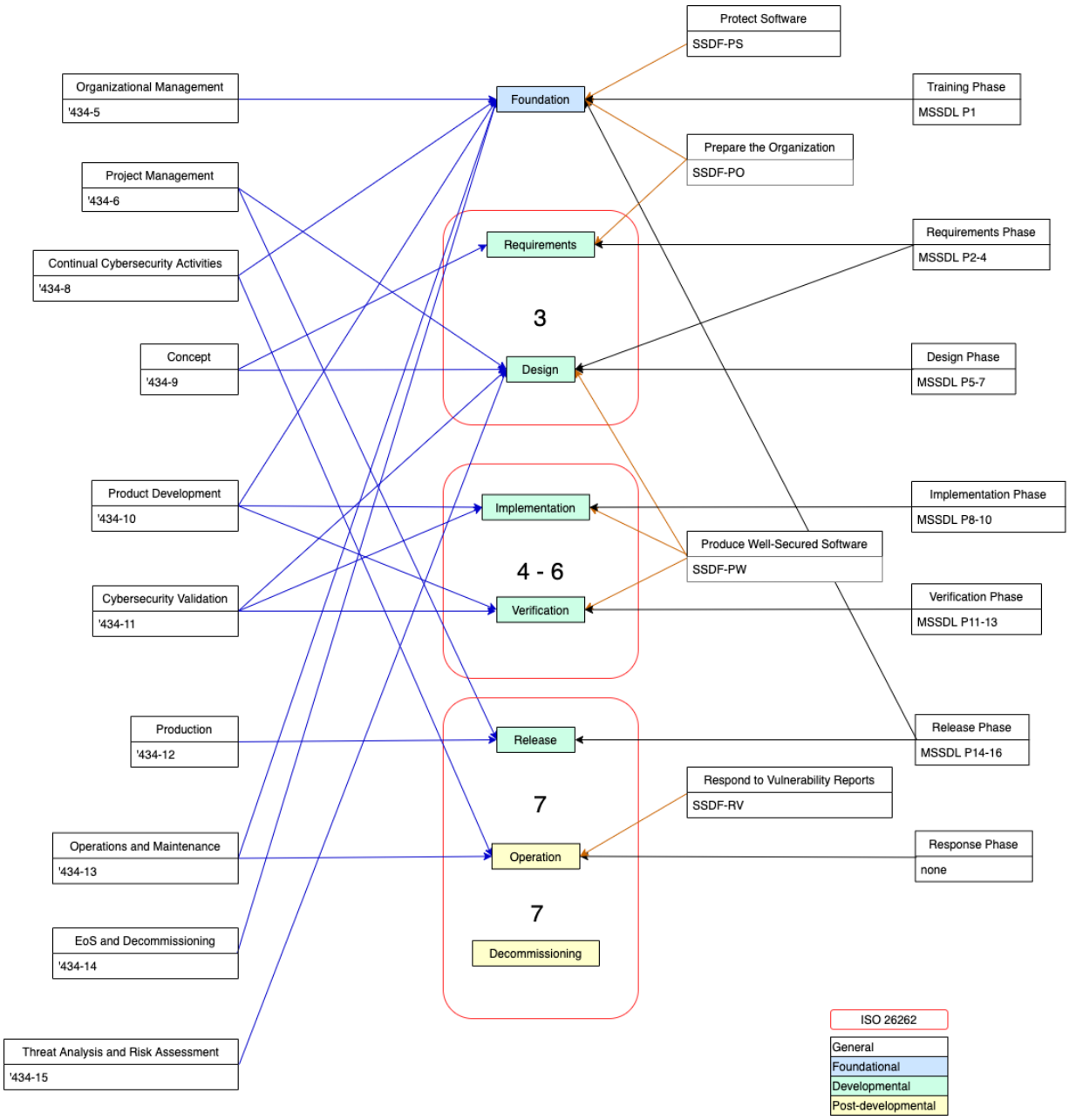

Figure 2 - AVCDL sources

Note: The red rounded rectangles indicate corresponding **ISO 26262** processes.

Note: Only major sources visualized.

4.3 Standards and Regulations Ecosystem

The underlying standards and regulations environment can be visualized as follows:

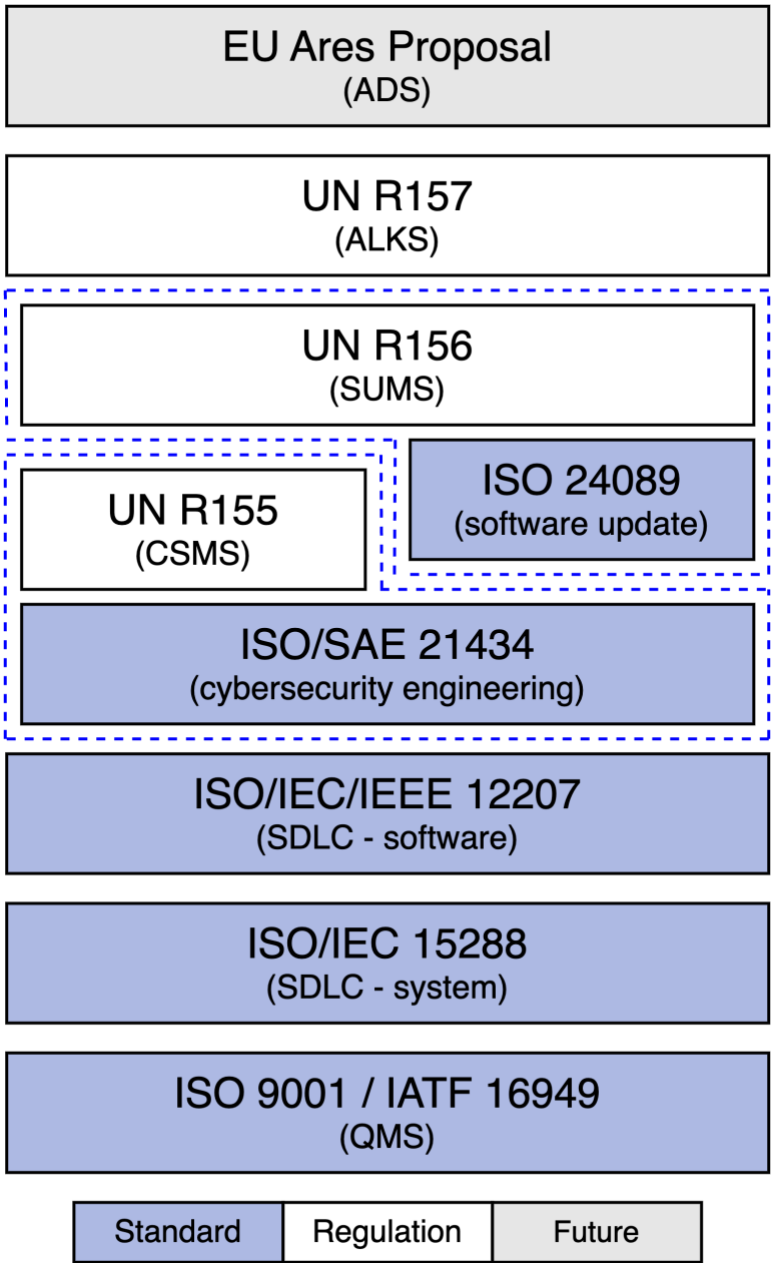

Figure 3 - Standards and Regulations Ecosystem (Automotive)

We can see here how the various standards build upon each other so that each layer is supported by the previous ones. Likewise, regulations are based on both standards and previously established regulations. For example, **UN R156** (Software Update Management System) is based on **UN R155** (CyberSecurity Management System) and **ISO 24089** (Road Vehicle – Software Updates Engineering), which is in turn dependent upon **ISO/SAE 21434** (Road Vehicle – Cybersecurity Engineering).

5. Continuous Improvement

The creation of secure software is not simply a programming endeavor. It begins with ensuring all team members understand how software and systems are made secure; requires the additional stages in the design and testing phase; and permeates all typical development practices.

As a system of systems, an autonomous vehicle will always be subject to variable-rate development. Each system within the system of systems is developed at a rate which may (and probably does) differ from that of other systems. That being the case, it is highly unlikely that the security profile of the system of systems will be the same as any individual system within it. This motivates the adoption of a process of continual improvement within each of the constituent systems, driving toward an ever-improved overall security profile.

Continuous improvement is not merely *pro forma*, but requisite. Whether we are considering the initial implementation of security or ongoing development, there will be the need to develop and refine the security model. This will need to be done at every level of the system (physical, network, protocol, and application) as well as across all the various sub-systems. Given the time-consuming nature of threat modeling, risk assessment, and threat mitigation; the only practical approach is to apply ever-increasing levels of security. This may manifest as an outside-in approach wherein the external attack surfaces are secured first with the interior system following; or by addressing the fundamental mechanisms by which data is managed within the system; or a combination of the two, dependent upon the maturity of the security in place in any given sub-system.

The ***AVCDL*** itself will also be subject to continuous improvement. This may manifest in changes to individual phases and their associated requirements. It may include changes in ownership of various phase requirements. There will always be new tools to consider in implementation of the ***AVCDL***.

To track progress of implementation of the ***AVCDL*** within the organization, **ISO 21827** Systems Security Engineering - Capability Maturity Model (**SSE-CMM** [13]) will be used as criteria for evaluation. Additionally, applicable elements of Cybersecurity Maturity Model Certification (**US DoD CMMC** [1]) level assignments of the requirements called out in ***Protecting Controlled Unclassified Information in Nonfederal Systems and Organizations*** (**NIST SP 800-171**) [9] are identified and tracked for each ***AVCDL*** phase requirement.

An example tracking report is shown ***here***.

6. Relationship to Standards and Regulations

As stated in the ***overview***, the ***AVCDL*** is a set of identified processes, requirements for those processes, generated products, and their mapping to the corresponding work products of various standards. This section will provide more detail about the relationship between the ***AVCDL*** and these standards.

6.1 Compliance versus Conformance

The ***AVCDL*** is designed to enable the organization to comply with the requirements of and enable the creation of products capable of satisfying the specifications for work products called by various certification standards. It does not conform to these standards in that it does not assume that any given certification standard's structure matches the form of any given organization's product development lifecycle framework, which encompasses development, safety, security and other needs. Any certification methodology with processes not conforming to the organization's actual development processes will be error-prone and unsuccessful.

6.2 Product Mapping

The ***AVCDL*** is designed to overlay an **ISO/IEC/IEEE 15288** / **ISO/IEC/IEEE 12207** lifecycle. As such, roughly fifty products are generated during any given product release's lifetime. The ***AVCDL*** provides mapping of these, which are the natural outcomes from implementation of cybersecurity best practices to their related standard work products. In this way, developmental friction is reduced as there is no expectation that individual contributors from any group (product management, risk, devops, development, …) be familiar with these standards and their details. This would, in fact, be seen as a negative as standards such as **ISO/SAE 21434** and **UN R155** are only the first of what will become many jurisdictional compliance standards needing to be complied with. By having a best practice lifecycle (***AVCDL***), we are more readily able to embrace compliance with regulatory standards as they appear. Additional details can be found in **AVCDL Phase Requirement Product ISO/SAE 21434 Work Product Fulfillment Summary**.

6.3 Interaction with ISO 26262

Although cybersecurity is mentioned several times within the **ISO 26262**, it is only formally addressed in clause 5.4.2.3 (**Safety culture – effective communication channels**) and annex E (**Guidance on potential interaction of functional safety with cybersecurity**) of part 2 (**Management of functional safety**). Given that annex E is an informative (non-normative) section, it contains no requirements or work products. There are multiple activities defined within the **AVCDL** that support the guidance given in annex E. Additional details can be found in **AVCDL Phase Requirement Product ISO 26262 Work Product Fulfillment Summary**.

7. Hardware-Software Relationship

Upon reading the ***AVCDL***, the question often arises as to why there is no specific reference to the hardware aspects of cybersecurity and how this relates to **ISO/SAE 21434** work products. This section will provide more detail about the relationship between the two, as well as the implications with respect to **ISO/SAE 21434**.

7.1 AVCDL is about Process

The ***AVCDL*** is at the core a framework supporting a collection of processes implementing a set of requirements. As noted in the ***background material***, it is built assuming the presence of the **ISO/IEC/IEEE 15288** and **ISO/IEC/IEEE 12207** standards within the organization. It further presumes that a best practices hardware-software development strategy akin the that described in **ISO 26262** (V-model). The phases and their requirements are sufficient to cover both hardware and software.

This is not to say that there is no consideration of hardware within the realm of cybersecurity as applied to the product's lifecycle. The cybersecurity requirements have explicit provision for hardware-specific requirements. Refer to the secondary document ***Security Requirements Taxonomy*** for additional details.

Any hardware-specific requirements are linked to specific product elements during the requirements phase as set out in the secondary document ***Element-level Security Requirements***.

7.2 ISO/SAE 21434 Compliance

The ***AVCDL*** is designed to enable the production of a supporting case for certification under various standards including **ISO/SAE 21434** and **UN R155, R156, R157**. **ISO/SAE 21434** has no hardware-specific work products or requirements. There is the desire that we be able to show that we apply cybersecurity to both hardware and software. This will be evidenced through application of cybersecurity concepts, goals and requirements using the processes and requirements set out in the ***AVCDL***.

8. Implementation Framework

8.1 Rectilinear Visualization

As indicated in the background section, the structure of the ***AVCDL*** is inspired by multiple standards and best practices documents.

It can be visualized as follows:

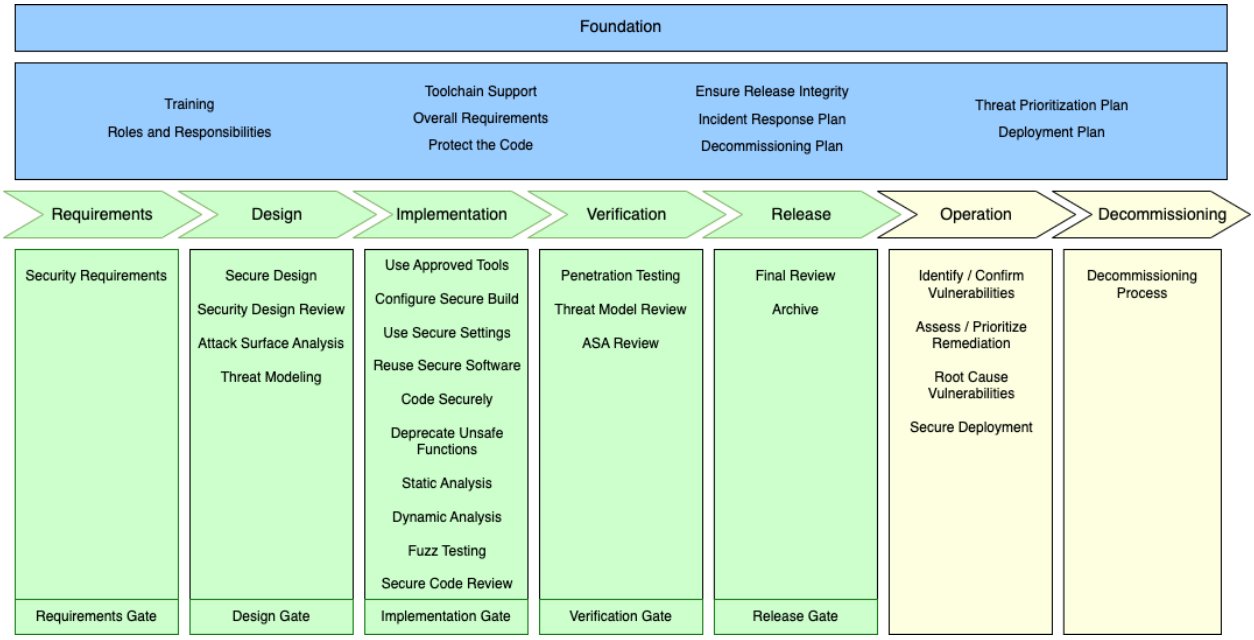

Figure 4 - AVCDL phases and requirements

Note: The rectilinear nature of this diagram is for convenience only and is not intended to imply any particular implementation or the use of any specific development methodology.

8.2 Cyclic Visualization

The ***AVCDL*** can also be visualized as a cyclic system (only phases shown):

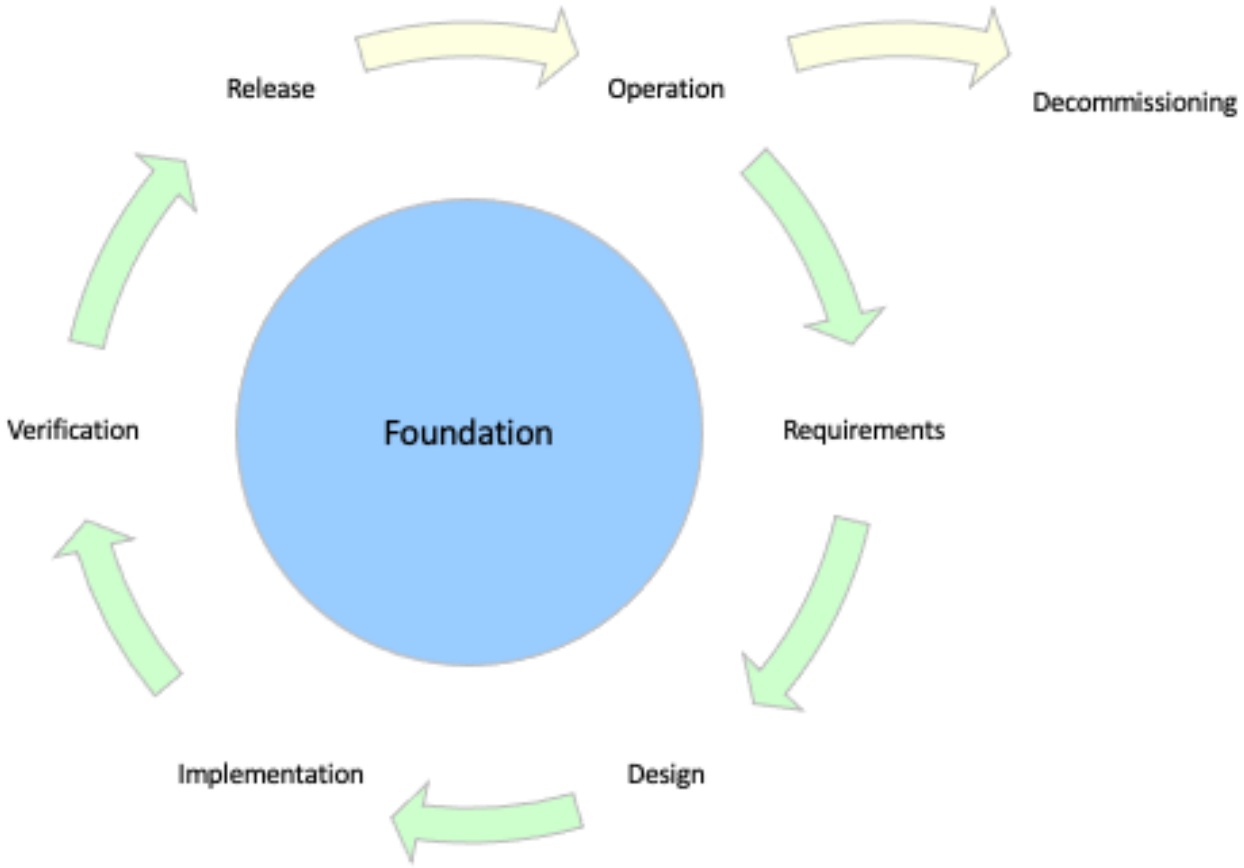

Figure 5 - AVCDL phases cyclic

8.3 Framework Categories

The process elements fall into three broad categories:

- **Foundational** process elements (shown in blue) form a foundation for secure development and take place outside the normal development cycle. These may be done in parallel and are subject to refresh as the security landscape changes. The basis for this phase comes from **MSSDL** and **ISO/SAE 21434**, and the practices from **SSDF**.

    - ***Foundation***

- **Intra-developmental** process elements (shown in green) serve to augment the existing development processes. The phases come from **MSSDL** and **ISO/SAE 21434**, and the practices from **SSDF**. To a large extent, the framework on which these phase augmentations hang already exists as part of the typical non-security-aware development process.

    - ***Requirements***
    - ***Design***
    - ***Implementation***
    - ***Verification***
    - ***Release***

    Note: In addition to its **ISO/SAE 21434**-supporting requirements, each of the intra-developmental phases has an exit gate requirement. There are no **ISO/SAE 21434** work products in the gate requirements as these are intended to verify the completion of the other requirements within the phase.

- **Post-developmental** phases (shown in yellow) take place once the product has been released.

    - ***Operation***
    - ***Decommissioning***

8.4 Implementation Methodology

It is important to note that the **AVCDL** does not mandate an implementation methodology (waterfall, XP, BDD, TDD, scrum, Kanban, spiral, …). The phases are dictated by the work product dependencies which are more fully explored in the **AVCDL Traceability** elaboration document.

8.4.1 Linear Methodologies

If a linear implementation methodology (waterfall, V-model, …) is employed, the phase diagram (***above***) can be used directly.

8.4.2 Cyclic Methodologies

If a cyclic implementation methodology (scrum-based agile, spiral, …) is employed, the intra-developmental phases overlay as follows:

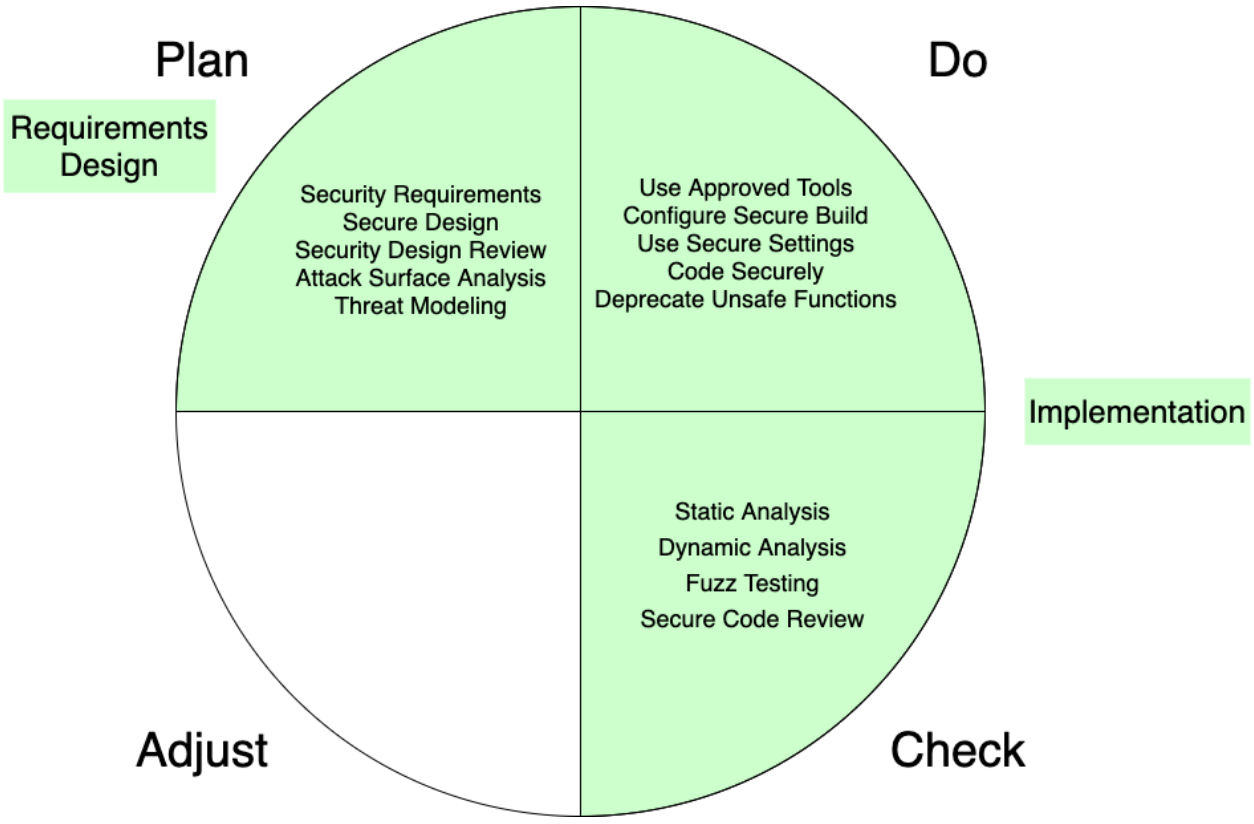

Figure 6 - AVCDL-PDCA Phase Requirement Mapping

Since cyclic implementation methodologies use short activity windows (sprints), there is a need to offset the security activities. The following diagram illustrates how the activity cycle unfolds into linear time.

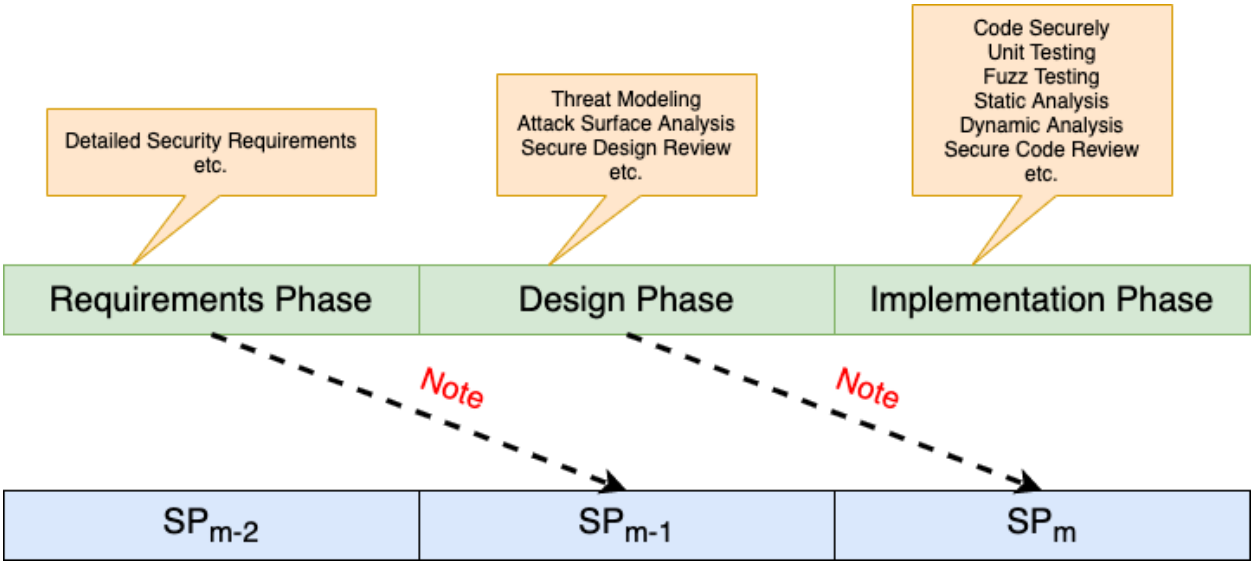

Figure 7 - Sprint-level Alignment

Note: The detailed requirements activity is performed one sprint in advance of design, and design one sprint in advance of implementation. A sprint SP_0 is assumed to bootstrap the process.

8.5 Metrics

In processes where quantitative data is gathered, it is useful to gather and track metrics. Metrics may be used to determine whether a process is being applied effectively to the system.

Processes include (but are not limited to):

- Attack surface analysis
- Threat modeling
- Fuzz testing
- Static analysis
- Dynamic analysis
- Penetration testing

Metrics may include (but are not limited to):

- Code coverage
- Issue lifetime
- Issue frequency
- Criticality

When metrics are used, it is important for the organization to establish appropriate scope and thresholds.

8.6 Availability of Products and Materials

It is presumed that all AVCDL phase products and the materials used in the generation thereof are available and granted appropriate access throughout the organization.

Materials may include (but are not limited to):

- Incident reports
- Interface agreements
- Requirements
- Databases (tools, components, …)

Products may include (but are not limited to):

- Reviews
- Reports
- Machine-readable intermediaries

8.7 Freshness of Products and Materials

The AVCDL has been designed to ensure that, if followed, its products and materials reflect the current state of the product under development. It achieves this through a collection of supporting mechanisms.

Note: Product and material freshness is ensured regardless of development implementation methodology.

These mechanisms include (but are not limited to):

- Human-readable reports generated from machine-readable activity products
- Uniform inter-process data interchange formats
- Modern standard data interchange formats
- Databases in place of individual text files
- Explicit activity failure behaviors
- Uniform threat prioritization and risk management across all activities
- Explicit verification phase review of design phase activities (threat modeling, attack surface analysis)
- Additional controls and their attendant testing being necessitated by additional knowledge of the system gained through ongoing cybersecurity activities

  Note: This type of feedback is illustrated in the **Incident Response Plan** and **Penetration Testing Report** secondary documents.

This topic is explored more extensively in the **Understanding Cybersecurity Risk Freshness in an AVCDL Context** elaboration document.

8.8 Defense-in-Depth

As discussed earlier, the AVCDL is intended to be implemented within the context of existing hardware, software and system lifecycles. It also addresses the two major deficiency areas in product cybersecurity, those being design and implementation. The following diagram shows the AVCDL phase requirements addressing each area.

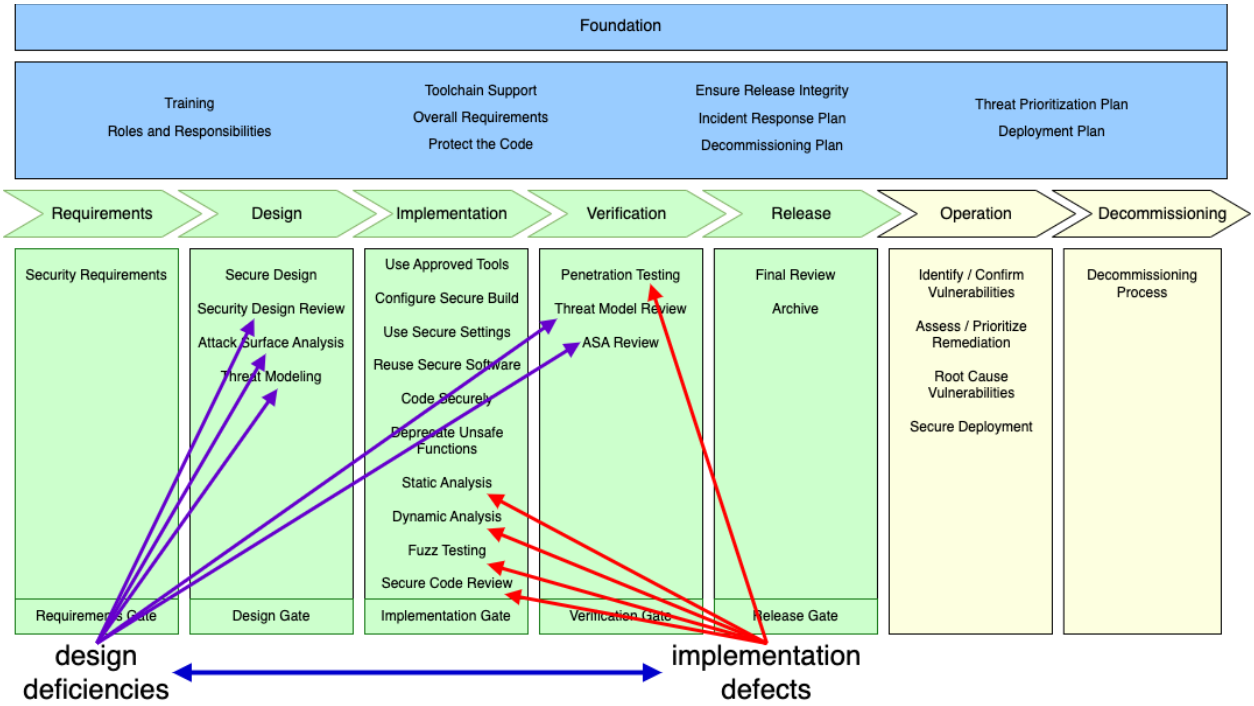

Figure 8 - Design vs Implementation Specific Requirements

Taken together these phase requirements provide overlapping coverage (defense-in-depth). It is presumed that in a mature organization the entirety of the AVCDL phase requirements will be implemented. Since every element of the product being created is intentionally subject to all of these phase requirements (subject to applicability), no consideration is given to the selection of one over another. In this way any variation in application that may come from individuals of different experience level is eliminated.

9. Process Phases and Requirements

The following is a summary of the AVCDL phases and their associated requirements.

- ***Foundation***
  - ***Foundation-1*** Training
  - ***Foundation-2*** Roles and Responsibilities
  - ***Foundation-3*** Toolchain Support
  - ***Foundation-4*** Definition of Security Requirements
  - ***Foundation-5*** Protect the Code
  - ***Foundation-6*** Ensure Release Integrity
  - ***Foundation-7*** Incident Response Plan
  - ***Foundation-8*** Decommissioning Plan
  - ***Foundation-9*** Threat Prioritization Plan
  - ***Foundation-10*** Deployment Plan

- ***Requirements***
  - ***Requirements-1*** Definition of Security Requirements
  - ***Requirements-2*** Requirements Gate

- ***Design***
  - ***Design-1*** Take Security Requirements and Risk Information into Account During Software Design
  - ***Design-2*** Review the Software Design to Verify Compliance with Security Requirements and Risk Information
  - ***Design-3*** Attack Surface Reduction
  - ***Design-4*** Threat Modeling
  - ***Design-5*** Design Gate

- ***Implementation***
  - ***Implementation-1*** Use Approved Tools
  - ***Implementation-2*** Configure the Compilation and Build Process to Improve Executable Security
  - ***Implementation-3*** Configure the Software to Have Secure Settings by Default
  - ***Implementation-4*** Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality
  - ***Implementation-5*** Create Source Code Adhering to Secure Coding Practice
  - ***Implementation-6*** Deprecate Unsafe Functions
  - ***Implementation-7*** Static Analysis
  - ***Implementation-8*** Dynamic Program Analysis
  - ***Implementation-9*** Security Code Review
  - ***Implementation-10*** Fuzz Testing
  - ***Implementation-11*** Implementation Gate
- ***Verification***
  - ***Verification-1*** Penetration Testing
  - ***Verification-2*** Threat Model Review
  - ***Verification-3*** Attack Surface Analysis Review
  - ***Verification-4*** Verification Gate
- ***Release***
  - ***Release-1*** Final Security Review
  - ***Release-2*** Archive
  - ***Release-3*** Release Gate
- ***Operation***
  - ***Operation-1*** Identify and Confirm Vulnerabilities on an Ongoing Basis
  - ***Operation-2*** Assess and Prioritize the Remediation of all Vulnerabilities
  - ***Operation-3*** Analyze Vulnerabilities to Identify Their Root Causes
  - ***Operation-4*** Secure Deployment
- ***Decommissioning***
  - ***Decommissioning-1*** Decommissioning Protocol
- ***Supplier Processes***
  - ***Supplier-1*** AVCMDS
  - ***Supplier-2*** Supplier Self-reported Maturity
  - ***Supplier-3*** Cybersecurity Interface Agreement

9.1 Foundation Phase

Predecessor: N/A

Successor: ***Requirements Phase***

These process elements form a foundation for secure development and take place outside the normal development cycle. These may be done in parallel and are subject to refresh as the security landscape changes.

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Foundation-1] Training

References: SSDF PO.1 / MSSDL P1

This training ensures that the **AVCDL** and its requirements are understood by those interacting with it.

[AVCDL-Foundation-2] Roles and Responsibilities

References: SSDF PO.2

It is critical to the success of any **AVCDL**-based project that the roles and responsibilities be defined and assigned prior to the phase to which they apply. These individuals serve as gatekeepers of security issues at the various phase gates.

[AVCDL-Foundation-3] Toolchain Support

References: SSDF PO.3 / MSSDL P8

Software supporting secure development must be evaluated, installed, and trained for.

[AVCDL-Foundation-4] Definition of Security Requirements

References: SSDF PO.4 / MSSDL P3

Before designing a secure system, it is necessary to have a clear and coherent set of security requirements.

These are the global security requirements as opposed to the more fine-grained requirements called out during the ***requirements phase***.

[AVCDL-Foundation-5] Protect the Code

References: SSDF PS.1

The code storage and access should be set up in such a way as to prevent inadvertent or intentional unauthorized changes, inappropriate access, or theft.

[AVCDL-Foundation-6] Ensure Release Integrity

References: SSDF PS.2

To some extent, this could be considered part of toolchain support. Operations such as code signing, and root-of-trust fall into this process element.

[AVCDL-Foundation-7] Incident Response Plan

References: MSSDL P14

The incident response plan covers the mechanisms needed for dealing with both internal and externally discovered security issues.

[AVCDL-Foundation-8] Decommissioning Plan

A framework must be in place for the eventual removal from service of an in-use system. This should cover the proper handling for any sensitive data embodied in the system.

[AVCDL-Foundation-9] Threat Prioritization Plan

References: MSSDL P3

A mechanism for quantifying potential risks and prioritizing their disposition must be established. This may take the form ranging from a gross-level quantization (bug bar) to a formal methodology (TARA).

[AVCDL-Foundation-10] Deployment Plan

A framework must be in place for the loading of software onto the system. This should include both the initial loading / configuration and updating. This should cover the proper handling for any sensitive data to be embodied in the system.

9.1.1 Training [AVCDL-Foundation-1]

Owner

Group: ***Security***

NCWF Role: ***Cyber Instructor***

Administration

security	devops	development	risk
R	I	C	I

There should be a general security awareness training covering the motivation for cybersecurity and its relationship to safety.

There are five distinct areas of training (as specified in ***MSSDL P1***):

- Secure design
- Threat modeling
- Secure coding
- Security testing
- Privacy

Aside from the awareness training, the other training classes have different target audiences. They may be presented concurrently. Ideally, they should be presented prior to the phase to which they apply. There should be an annual limited-scope refresher for each.

A training path is provided within the training material itself.

There is also the need to track individual and aggregate training participation.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	**none**
Development	List of programming languages / compilers
Risk	**none**

AVCDL Products

[AVCDL-Foundation-1.1] ***Training Catalog***

[AVCDL-Foundation-1.2] ***System to Track Training Participation***

9.1.2 Roles and Responsibilities [AVCDL-Foundation-2]

Owner

Group: ***Security***

NCWF Role: ***Systems Requirements Planner***

Administration

security	devops	development	risk
R	C	C	-

NIST SP 800-181 [***National Initiative for Cybersecurity Education (NICE) Cybersecurity Workforce Framework (NCWF)***] provides an exhaustive breakdown of cybersecurity roles and responsibilities. It provides a common, consistent lexicon that categorizes and describes cybersecurity work. We will draw upon these to establish those needed in support of the ***AVCDL***.

Note: Additional information on ***NCWF*** can be found on their ***site***.

Note: There will be tasks and abilities called out for roles in NCWF which are not leveraged. Additionally, there will be areas where there is not a 1-to-1 mapping.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

none

AVCDL Products

[AVCDL-Foundation-2.1] ***Roles and Responsibilities Document***

9.1.3 Toolchain Support [AVCDL-Foundation-3]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

All software used in the product's development (tools and code [as source or binaries]) must be vetted by development, cybersecurity, and legal to ensure that it is appropriate for use in the development of safety-related systems. A catalog of this software should be created for use by later processes.

The following additional software needs to be in place to support secure development:

- threat modeling
- attack surface analysis
- compile-time security analysis
- static security analysis (including MISRA and SEI CERT)
- open source and third-party software tracking
- security incident tracking

Note: Training for each must be provided prior to time of use.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	Component tracking system
Development	List of development tools
Risk	**none**

AVCDL Products

[AVCDL-Foundation-3.1] ***List of Approved Tools and Components***

9.1.4 Definition of Security Requirements [AVCDL-Foundation-4]

Owner

Group: ***Security***

NCWF Role: ***Systems Requirements Planner***

Administration

security	devops	development	risk
R	I	I	-

Before designing a secure system, it is necessary to have a clear and coherent set of security requirements.

These are the global security requirements as opposed to the more fine-grained requirements called out during the ***requirements phase***.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

none

AVCDL Products

[AVCDL-Foundation-4.1] ***Global Security Goals***
[AVCDL-Foundation-4.2] ***Global Security Requirements***

9.1.5 Protect the Code [AVCDL-Foundation-5]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	-	-

Processes and controls need to be in place to accomplish the following goals:

- secure code storage
- IP / open source / third party material segregation
- Deterministic builds
- Disaster recovery
- Security audit

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	Secure IT infrastructure
Development	**none**
Risk	**none**

AVCDL Products

[AVCDL-Foundation-5.1] ***Code Protection Plan***

9.1.6 Ensure Release Integrity [AVCDL-Foundation-6]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

Steps need to be in place to accomplish the following tasks:

- code signing
- hash tracking
- credential management
- root-of-trust
- secure deployment support

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	• Code signing • Credential management • Deployment infrastructure
Development	**none**
Risk	**none**

AVCDL Products

[AVCDL-Foundation-6.1] ***Release Integrity Plan***

9.1.7 Incident Response Plan [AVCDL-Foundation-7]

Owner

Group: ***Security***

NCWF Role: ***Partner Integration Planner***

Administration

security	devops	development	risk
R	-	C	-

Monitoring sources may include:

- external sources
  - government sources
  - commercial or non-commercial sources
  - researchers
  - organization's supply chain
  - organization's customers
- internal sources
  - vulnerability analysis results
  - information from the field (vulnerability scanning reports, repair information, consumer usage information)

The incident response plan should include:

- An identified sustained engineering (SE)
- On-call contacts with decision-making authority 24/7/365
- Security servicing plans for both issues related to internally and externally supplied software

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Triage information required
Risk	**none**

AVCDL Products

[AVCDL-Foundation-7.1] ***Cybersecurity Monitoring Plan***

[AVCDL-Foundation-7.2] ***Incident Response Plan***

9.1.8 Decommissioning Plan [AVCDL-Foundation-8]

Owner

Group: ***Security***

NCWF Role: ***Partner Integration Planner***

Administration

security	devops	development	risk
R	C	C	-

The decommissioning plan covers the proper handling for any sensitive data embodied in the system. This includes:

- credentials
- certificates
- PII
- logs

It is critical that the decommissioning plan include partner elements within the system.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	Decommissioning / RMA process
Development	List of data stored on systems
Risk	**none**

AVCDL Products

[AVCDL-Foundation-8.1] ***Decommissioning Plan***

9.1.9 Threat Prioritization Plan [AVCDL-Foundation-9]

Owner

Group: ***Security***

NCWF Role: ***Systems Requirements Planner***

Administration

security	devops	development	risk
R	-	I	R

A mechanism for quantifying potential risks and prioritizing their disposition must be established. This may take the form from a gross-level quantization (bug bar) to a formal methodology (TARA).

Since an autonomous vehicle is a safety-critical system, a formal threat quantification system is warranted.

This plan will be applied to take a threat potential (output of the threat modeling process) and yield a prioritized set of threat issues which must be addressed in order to ensure the safe operation of the vehicle.

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

none

AVCDL Products

[AVCDL-Foundation-9.1] ***Threat Prioritization Plan***

9.1.10 Deployment Plan [AVCDL-Foundation-10]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

The deployment framework must consider the following:

- secure software deployment
- initial and update scenarios
- deployment failure handling

The deployment plan covers the proper handling for any sensitive data embodied in the system. This includes:

- credentials
- certificates
- PII
- logs

Training Provided

none

Phase Requirement Dependencies

none

External Group Dependencies

Group	Inputs
Devops	Deployment infrastructure / process
Development	List of material to be deployed
Risk	**none**

AVCDL Products

[AVCDL-Foundation-10.1] ***Deployment Plan***

9.2 Requirements Phase

Predecessor: ***Foundation Phase*** or ***Operation Phase***

Successor: ***Design Phase***

The requirements phase of development is a reiteration of ***AVCDL-Foundation-4*** **Definition of Security Requirements** but with higher resolution. In an Agile-based development process, this is to be expected.

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Requirements-1] Security Requirements Definition

References: SSDF PO.4 / MSSDL P2

The requirements are created with consideration of the global security requirements. They provide constraints specific to the work under consideration.

[AVCDL-Requirements-2] Requirements Gate

References: MSSDL P3

Requirements phase exit is conditional (formally gated) on completion of all AVCDL phase requirements and work products for this phase.

9.2.1 Security Requirements Definition [AVCDL-Requirements-1]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	I	-

Requirements need to both consider the global security requirements and add constraints necessary to the specifics of the work under consideration. As with the global-level requirements called out in ***AVCDL-Foundation-4***, these requirements should be derived using the ***security requirements taxonomy*** in order to expose gaps up-front (prior to threat modeling, attack surface analysis, ...).

Requirements should be traceable through the product operation phase to allow for improvement should deficiencies be discovered.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-4] Definition of Security Requirements

External Group Dependencies

Group	Inputs
Devops	**none**
Development	High-level design
Risk	**none**

AVCDL Products

[AVCDL-Requirements-1.1] ***Element-level Security Goals***

[AVCDL-Requirements-1.2] ***Element-level Security Requirements***

9.2.2 Requirements Gate [AVCDL-Requirements-2]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	-	R	-

Requirements phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase. The security advisor assigned to the release must certify that the project team has satisfied security requirements for this phase.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Requirements-1] Security Requirements Definition

External Group Dependencies

none

AVCDL Products

[AVCDL-Requirements-2.1] ***Requirements Phase Gate***

9.3 Design Phase

Predecessor: ***Requirements Phase***

Successor: ***Implementation Phase***

The changes to the design phase include the incorporation of security requirements and analysis of the design from a security perspective.

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Design-1] Apply Security Requirements and Risk Information to Design

References: SSDF PW.1 / MSSDL P4 / MSSDL P5

The design should take into consideration established security requirements and risk information.

[AVCDL-Design-2] Security Design Review

References: SSDF PW.2

Help ensure the software will meet the security requirements and satisfactorily address the identified risk information.

[AVCDL-Design-3] Attack Surface Reduction

References: MSSDL P6

Attack surface analysis guides the disabling or access restricting of system services. It applies the principles of least privilege and layered defense.

[AVCDL-Design-4] Threat Modeling

References: MSSDL P7

Threat modeling realizes an abstraction of the system as a set of interacting processes managing resources passing data between them. It is on these data flows that automated threat modeling tools reason.

[AVCDL-Design-5] Design Gate

References: MSSDL P3

Design phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase.

9.3.1 Apply Security Requirements and Risk Information to Design [AVCDL-Design-1]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
R	-	R	-

Determine which security requirements the software's design should meet and determine what security risks the software is likely to face during production operation and how those risks should be mitigated by the software's design. Addressing security requirements and risks during software design instead of later helps to make software development more efficient.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Requirements-2] Requirements Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Detailed functional requirements
Risk	**none**

AVCDL Products

[AVCDL-Design-1.1] ***Design Showing Security Considerations***

9.3.2 Security Design Review [AVCDL-Design-2]

Owner

Group: ***Security***

NCWF Role: ***Systems Requirements Planner***

Administration

security	devops	development	risk
R	-	R	C

Help ensure the software will meet the security requirements and satisfactorily address the identified risk information.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-1] Apply Security Requirements and Risk Information to Design

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Design-2.1] ***Security Design Review Report***

9.3.3 Attack Surface Reduction [AVCDL-Design-3]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	R	-

Attack surface reduction encompasses shutting off or restricting access to system services, applying the principle of least privilege, and employing layered defenses wherever possible. It is primarily used when dealing with externally supplied elements where access to the design is not provided.

Note: Attack surface analysis is a methodology which trails in maturity when compared with threat modeling. Automated measures in this area will be limited.

Note: The attack surface analysis **AVCDL** work products are generated through application of the threat prioritization plan set out in ***[AVCDL-Foundation-9]*** **Threat Prioritization Plan**.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-1] Apply Security Requirements and Risk Information to Design

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Functional OS interface design
Risk	**none**

AVCDL Products

[AVCDL-Design-3.1] ***Attack Surface Analysis Report***
[AVCDL-Design-4.2] ***Ranked / Risked Threat Report***
[AVCDL-Design-4.3] ***Threat Report***

9.3.4 Threat Modeling [AVCDL-Design-4]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	R	-

Threat modeling is an exercise which may be done at any stage of development. It realizes an abstraction of the system as a set of interacting processes managing resources passing data between them. It is on these data flows that automated threat modeling tools reason.

In that same way that security requirements should be considered at multiple levels in order to provide a complete landscape, so to do threat models.

Note: Threat modeling is a team exercise, encompassing program/project managers, developers, and testers, and represents the primary security analysis task performed during the software design stage.

Note: The threat modeling **AVCDL** work products are generated through application of the threat prioritization plan set out in ***[AVCDL-Foundation-9]*** **Threat Prioritization Plan**.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-9] Threat Prioritization Plan
[AVCDL-Design-1] Apply Security Requirements and Risk Information to Design

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Design-4.1] ***Threat Modeling Report***

[AVCDL-Design-4.2] ***Ranked / Risked Threat Report***

[AVCDL-Design-4.3] ***Threat Report***

9.3.5 Design Gate [AVCDL-Design-5]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	-	R	R

Design phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase. The security advisor assigned to the release must certify that the project team has satisfied security requirements for this phase.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Design-2]	Security Design Review
[AVCDL-Design-3]	Attack Surface Reduction
[AVCDL-Design-4]	Threat Modeling

External Group Dependencies

Group	Inputs
Devops	**none**
Development	**none**
Risk	**none**

AVCDL Products

[AVCDL-Design-5.1] ***Design Phase Gate***

9.4 Implementation Phase

Predecessor: ***Design Phase***

Successor: ***Verification Phase***

The implementation phase of development is based on **MSSDL Implementation Phase** (MSSDL P8-10) and **SSDF Prepare Well-Secured Software** (PW.4-7, 9).

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Implementation-1] Use Approved Tools

References: MSSDL P8

Development teams should strive to use the latest version of approved tools to take advantage of new security analysis functionality and protections.

[AVCDL-Implementation-2] Configure Build Process to Improve Security

References: SSDF PW.6

Decrease the number of security vulnerabilities in the software and reduce costs by eliminating vulnerabilities before testing occurs.

[AVCDL-Implementation-3] Use Secure Settings by Default

References: SSDF PW.9

Help improve the security of the software at installation time, which reduces the likelihood of the software being deployed with weak security settings that would put it at greater risk of compromise.

[AVCDL-Implementation-4] Reuse Well-Secured Software

References: SSDF PW.4

Reuse of well-secured (verified) software lowers the costs of development, expedites development, and decreases the likelihood of introducing additional security vulnerabilities.

[AVCDL-Implementation-5] Code Securely

References: SSDF PW.5

Decrease the number of security vulnerabilities in the software and reduce costs by eliminating vulnerabilities during source code creation.

[AVCDL-Implementation-6] Deprecate Unsafe Functions

References: MSSDL P9

Project teams should analyze all functions and APIs that will be used in conjunction with a software development project and prohibit those that are determined to be unsafe.

[AVCDL-Implementation-7] Static Analysis

References: MSSDL P10

Project teams should perform static analysis of source code.

[AVCDL-Implementation-8] Dynamic Program Analysis

References: MSSDL P11

Run-time verification of software programs is necessary to ensure that a program's functionality works as designed.

[AVCDL-Implementation-9] Security Code Review

References: SSDF PW.7

The security team and security advisors should augment static analysis with other automated or human review as appropriate.

[AVCDL-Implementation-10] Fuzz Testing

References: MSSDL P12

Fuzz testing is a specialized form of dynamic analysis used to induce program failure by deliberately introducing malformed or random data to an application.

[AVCDL-Implementation-11] Implementation Gate

References: MSSDL P3

Implementation phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase.

9.4.1 Use Approved Tools [AVCDL-Implementation-1]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
C	C	R	-

Development teams should strive to use the latest version of **approved** tools and components to take advantage of new security analysis functionality and protections. Unapproved tools and components should never be used. The build system should verify that the tools and components currently being used have been approved for use in the creation of this product.

Note: The list of approved tools and components, and their associated security checks, such as compiler and linker options, and warnings were created in the ***foundation phase***.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Foundation-3] Toolchain Support
[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	Component tracking comparison system
Development	**none**
Risk	**none**

AVCDL Products

[AVCDL-Implementation-1.1] ***List of Tools and Components Used***

9.4.2 Configure Build Process to Improve Security [AVCDL-Implementation-2]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

Decrease the number of security vulnerabilities in the software and reduce costs by eliminating vulnerabilities before testing occurs.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	Build system
Development	List of adopted secure build settings
Risk	**none**

AVCDL Products

[AVCDL-Implementation-2.1] ***Build Process Documentation***

9.4.3 Use Secure Settings by Default [AVCDL-Implementation-3]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	R	-

Helps improve the security of the software at installation time, which reduces the likelihood of the software being deployed with weak security settings that would put it at greater risk of compromise.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Implementation-3.1] ***Secure Settings Document***

9.4.4 Reuse Well-Secured Software [AVCDL-Implementation-4]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
C	I	R	-

Lower the costs of software development, expedite software development, and decrease the likelihood of introducing additional security vulnerabilities into the software. These are particularly true for software that implements security functionality, such as cryptographic modules and protocols.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	List of libraries used
Risk	**none**

AVCDL Products

[AVCDL-Implementation-4.1] ***Component / Version - Product / Version Cross-reference Document***

9.4.5 Code Securely [AVCDL-Implementation-5]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
C	-	R	-

Decrease the number of security vulnerabilities in the software and reduce costs by eliminating vulnerabilities during source code creation.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element implementation
Risk	**none**

AVCDL Products

[AVCDL-Implementation-5.1] ***Secure Development***

9.4.6 Deprecate Unsafe Functions [AVCDL-Implementation-6]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
C	-	R	-

Many commonly used functions and APIs are not secure in the face of the current threat environment. Project teams should analyze all functions and APIs that will be used in conjunction with a software development project and prohibit those that are determined to be unsafe.

Note: The list of unsafe functions should have been created in the ***foundation phase***.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	List of deprecated functions in use
Risk	**none**

AVCDL Products

[AVCDL-Implementation-6.1] ***Currently Used Deprecated Functions Document***

9.4.7 Static Analysis [AVCDL-Implementation-7]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

Project teams should perform static analysis of source code.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	• Static analysis infrastructure • Static analysis settings tracking
Development	List of adopted security-related settings
Risk	**none**

AVCDL Products

[AVCDL-Implementation-7.1] ***Static Analysis Report***

9.4.8 Dynamic Program Analysis [AVCDL-Implementation-8]

Owner

Group: ***Development***

NCWF Role: ***Software Developer***

Administration

security	devops	development	risk
C	-	R	-

Run-time verification of software programs is necessary to ensure that a program's functionality works as designed. This verification task should specify tools that monitor application behavior for memory corruption, user privilege issues, and other critical security problems.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	Dynamic analysis testing infrastructure
Development	List of adopted security-related tools
Risk	**none**

AVCDL Products

[AVCDL-Implementation-8.1] ***Dynamic Analysis Report***

9.4.9 Security Code Review [AVCDL-Implementation-9]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	-	C	-

Static code analysis by itself is generally insufficient to replace a manual code review. The security team and security advisors should be aware of the strengths and weaknesses of static analysis tools and be prepared to augment static analysis tools with other tools or human review as appropriate.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	Code review infrastructure
Development	Element implementation
Risk	**none**

AVCDL Products

[AVCDL-Implementation-9.1] ***Secure Code Review Summary***

9.4.10 Fuzz Testing [AVCDL-Implementation-10]

Owner

Group: ***Security***

NCWF Role: ***Vulnerability Assessment Analyst***

Administration

security	devops	development	risk
R	C	C	-

Fuzz testing is a specialized form of dynamic analysis used to induce program failure by deliberately introducing malformed or random data to an application. The fuzz testing strategy is derived from the intended use of the application and the functional and design specifications for the application.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Design-5] Design Gate

External Group Dependencies

Group	Inputs
Devops	Fuzz testing process infrastructure
Development	Element implementation
Risk	**none**

AVCDL Products

[AVCDL-Implementation-10.1] ***Fuzz Testing Report***

9.4.11 Implementation Gate [AVCDL-Implementation-11]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	R	R	-

Implementation phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase. The security advisor assigned to the release must certify that the project team has satisfied security requirements for this phase.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Implementation-1]	Use Approved Tools
[AVCDL-Implementation-2]	Configure Build Process to Improve Security
[AVCDL-Implementation-3]	Use Secure Settings by Default
[AVCDL-Implementation-4]	Reuse Well-Secured Software
[AVCDL-Implementation-5]	Code Securely
[AVCDL-Implementation-6]	Deprecate Unsafe Functions
[AVCDL-Implementation-7]	Static Analysis
[AVCDL-Implementation-8]	Dynamic Program Analysis
[AVCDL-Implementation-9]	Security Code Review
[AVCDL-Implementation-10]	Fuzz Testing

External Group Dependencies

none

AVCDL Products

[AVCDL-Implementation-11.1] ***Implementation Phase Gate***

9.5 Verification Phase

Predecessor: ***Implementation Phase***

Successor: ***Release Phase***

The verification phase of development is based on **MSSDL Verification Phase** (MSSDL P11-3) and **SSDF Produce Well-Secured Software** (SSDF PW.8).

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Verification-1] Penetration Testing

References: SSDF PW.8

Penetration testing identifies vulnerabilities before software is released so they can be corrected before release, which prevents exploitation.

[AVCDL-Verification-2] Threat Model Review

References: MSSDL P13

The threat models should be reviewed to ensures that any design or implementation changes to the system have been accounted for, and that any new attack vectors created as a result of the changes have been reviewed and mitigated.

[AVCDL-Verification-3] Attack Surface Analysis Review

References: MSSDL P13

The attack surface analysis should be reviewed to ensures that any design or implementation changes to the system have been accounted for, and that any new attack vectors created as a result of the changes have been reviewed and mitigated.

[AVCDL-Verification-4] Verification Gate

References: MSSDL P3

Verification phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase.

9.5.1 Penetration Testing [AVCDL-Verification-1]

Owner

Group: ***Security***

NCWF Role: ***Vulnerability Assessment Analyst***

Administration

security	devops	development	risk
R	C	C	-

Help identify vulnerabilities before software is released so they can be corrected before release, which prevents exploitation. Using automated methods lowers the effort and resources needed to detect vulnerabilities. Executable code is binaries, directly executed bytecode, directly executed source code, and any other form of code an organization deems as executable.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Implementation-11] Implementation Gate

External Group Dependencies

Group	Inputs
Devops	Penetration testing process infrastructure
Development	Operational system
Risk	**none**

AVCDL Products

[AVCDL-Verification-1.1] ***Penetration Testing Report***

[AVCDL-Design-4.2] ***Ranked / Risked Threat Report***

[AVCDL-Design-4.3] ***Threat Report***

9.5.2 Threat Model Review [AVCDL-Verification-2]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	R	-

The threat models should be reviewed to ensure that any design or implementation changes to the system have been accounted for, and that any new attack vectors created as a result of the changes have been reviewed and mitigated.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Design-4] Threat Modeling
[AVCDL-Implementation-11] Implementation Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Updated element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Verification-2.1] ***Updated Threat Model***

9.5.3 Attack Surface Analysis Review [AVCDL-Verification-3]

Owner

Group: ***Security***

NCWF Role: ***Security Architect***

Administration

security	devops	development	risk
R	-	R	-

The attack surface analysis should be reviewed to ensure that any design or implementation changes to the system have been accounted for, and that any new attack vectors created as a result of the changes have been reviewed and mitigated.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Design-3]	Attack Surface Reduction
[AVCDL-Implementation-11]	Implementation Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Updated functional OS interface design
Risk	**none**

AVCDL Products

[AVCDL-Verification-3.1] ***Updated Attack Surface Analysis***

9.5.4 Verification Gate [AVCDL-Verification-4]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	-	R	R

Verification phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase. The security advisor assigned to the release must certify that the project team has satisfied security requirements for this phase.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Verification-1]	Penetration Testing
[AVCDL-Verification-2]	Threat Model Review
[AVCDL-Verification-3]	Attack Surface Analysis Review

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Updated element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Verification-4.1] ***Verification Phase Gate***

9.6 Release Phase

Predecessor: ***Verification Phase***

Successor: ***Operation Phase***

The release phase of development is based on **MSSDL Release Phase** (MSSDL P14-6).

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Release-1] Final Security Review

References: MSSDL P15

The Final Security Review (FSR) is a deliberate examination of all the security activities performed on a software application prior to release.

[AVCDL-Release-2] Archive

References: MSSDL P16

All pertinent information and data must be archived to allow for post-release servicing of the software.

[AVCDL-Release-3] Release Gate

References: MSSDL P16

Release phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase.

9.6.1 Final Security Review [AVCDL-Release-1]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	C	C	C

This is a deliberate examination of all the security activities performed on a software application prior to release. The FSR is performed by the security advisor with assistance from the regular development staff and the security and privacy team leads. The FSR is not a “penetrate and patch” exercise, nor is it a chance to perform security activities that were previously ignored or forgotten.

The FSR usually includes an examination of:

- threat models
- exception requests
- tool output
- performance reports

These are compared against the previously determined quality gates or bug bars. Regressions discovered at this stage indicate a failure in the verification phase.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Verification-4] Verification Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Final design documentation
Risk	**none**

AVCDL Products

[AVCDL-Release-1.1] ***Final Security Review Report***

9.6.2 Archive [AVCDL-Release-2]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
-	R	C	-

Everything necessary to reproduce and maintain the product must be archived.

This includes:

- specifications
- source code
- binaries
- private symbols
- threat models
- documentation
- emergency response plans
- license and servicing terms for any third-party software
- other data necessary to perform post-release servicing tasks

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Release-1] Final Security Review

External Group Dependencies

Group	Inputs
Devops	• Artifact storage infrastructure • Artifact tracking system
Development	Final materials for deployment
Risk	**none**

AVCDL Products

[AVCDL-Release-2.1] ***Archive Manifest***

9.6.3 Release Gate [AVCDL-Release-3]

Owner

Group: ***Security***

NCWF Role: ***Secure Software Assessor***

Administration

security	devops	development	risk
R	R	R	R

Release phase exit is conditional (formally gated) on completion of all **AVCDL** phase requirements and work products for this phase. The security advisor assigned to the release must certify that the project team has satisfied security requirements.

Training Provided

none

Phase Requirement Dependencies

[AVCDL-Release-2] Archive

External Group Dependencies

none

AVCDL Products

[AVCDL-Release-3.1] ***Release Phase Gate***

9.7 Operation Phase

Predecessor: ***Release Phase***

Successor: ***Requirements Phase*** or ***Decommissioning Phase***

The operation phase is based on **SSDF Vulnerability Report Practices** (SSDF RV).

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Operation-1] Identify and Confirm Vulnerabilities

References: SSDF RV.1

Help ensure vulnerabilities are identified more quickly so they can be remediated more quickly, reducing the window of opportunity for attackers.

[AVCDL-Operation-2] Assess and Prioritize the Remediation

References: SSDF RV.2

Help ensure vulnerabilities are remediated as quickly as necessary, reducing the window of opportunity for attackers.

[AVCDL-Operation-3] Root Cause Vulnerabilities

References: SSDF RV.3

Help reduce the frequency of vulnerabilities in the future.

[AVCDL-Operation-4] Secure Deployment

Software must be deployed in a secure manner.

9.7.1 Identify and Confirm Vulnerabilities [AVCDL-Operation-1]

Owner

Group: ***Security***

NCWF Role: ***Cyber Defense Incident Responder***

Administration

security	devops	development	risk
R	-	C	-

Help ensure vulnerabilities are identified more quickly so they can be remediated more quickly, reducing the window of opportunity for attackers.

Note: The incident response report **AVCDL** work product is generated through application of the threat prioritization plan set out in ***[AVCDL-Foundation-9]*** **Threat Prioritization Plan**.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-7] Incident Response Plan
[AVCDL-Release-3] Release Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element detailed design
Risk	**none**

AVCDL Products

[AVCDL-Operation-1.1] ***Cybersecurity Incident Report***

9.7.2 Assess and Prioritize Remediation [AVCDL-Operation-2]

Owner

Group: ***Security***

NCWF Role: ***Cyber Defense Forensics Analyst***

Administration

security	devops	development	risk
R	-	C	C

Help ensure vulnerabilities are remediated as quickly as necessary, reducing the window of opportunity for attackers.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-7] Incident Response Plan
[AVCDL-Release-3] Release Gate

External Group Dependencies

none

AVCDL Products

[AVCDL-Operation-1.1] ***Cybersecurity Incident Report***

9.7.3 Root Cause Vulnerabilities [AVCDL-Operation-3]

Owner

Group: ***Security***

NCWF Role: ***Cyber Defense Forensics Analyst***

Administration

security	devops	development	risk
R	-	C	-

Help reduce the frequency of vulnerabilities in the future.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-7] Incident Response Plan
[AVCDL-Release-3] Release Gate

External Group Dependencies

Group	Inputs
Devops	**none**
Development	Element implementation
Risk	**none**

AVCDL Products

[AVCDL-Operation-1.1] ***Cybersecurity Incident Report***

9.7.4 Secure Deployment [AVCDL-Operation-4]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
C	R	C	-

Software must be deployed in a secure manner.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-10] Deployment Plan
[AVCDL-Release-3] Release Gate

External Group Dependencies

Group	Inputs
Devops	• Deployment infrastructure • Deployment process
Development	Materials for deployment
Risk	**none**

AVCDL Products

[AVCDL-Operation-4.1] ***Software Deployment Report***

9.8 Decommissioning Phase

Predecessor: ***Operation Phase***

Successor: N/A

Decommissioning is a part of the lifecycle of an item or component and is considered in the concept and product development phases.

Decommissioning is different from end of support. An organization can end support for an item or component, but that item or component can still function as designed in the field. Both decommissioning and end of support present cybersecurity implications, but those implications are considered separately.

Every product release should include a ***decommissioning plan*** containing information as to how to properly dispose of the security-related information constrained within the product.

Note: The product dependencies for this phase are visualized in the **AVCDL Traceability** elaboration document.

[AVCDL-Decommissioning-1] Apply Decommissioning Protocol

The decommissioning protocol specified in the decommissioning plan should be applied to the system coming out of service.

9.8.1 Apply Decommissioning Protocol [AVCDL-Decommissioning-1]

Owner

Group: ***Devops***

NCWF Role: ***Information Systems Security Developer***

Administration

security	devops	development	risk
I	R	-	-

Apply protocols appropriate to ensuring that any and all security-related information has been purged from the system.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-8] Decommissioning Plan

External Group Dependencies

Group	Inputs
Devops	**none**
Development	List of data stored on systems
Risk	**none**

AVCDL Products

[AVCDL-Decommissioning-1.1] ***Decommissioning Report***

9.9 Supplier Processes

Predecessor: N/A

Successor: N/A

Although the supplier processes are primarily the responsibility of the organization, there is a considerable amount of support required to ensure that the cybersecurity aspects of the distributed development are addressed.

Note: As the supplier processes are not primary to the AVCDL, they do not appear in roll-ups or traceability maps.

Note: A training path for supplier processes is provided within the supplier training material.

[AVCDL-Supplier-1] AVCMDS

The Autonomous Vehicle Cybersecurity Manufacturer Disclosure Statement (AVCMDS) details the practices and capabilities of a supplier.

[AVCDL-Supplier-2] Supplier Self-reported Maturity

Supplier self-report maturity serves as a basis for establishing the cybersecurity relationship between the supplier and customer.

[AVCDL-Supplier-3] Cybersecurity Interface Agreement

The Cybersecurity Interface Agreement (CIA) legally establishes the relationship between the supplier and customer.

9.9.1 AVCMDS [AVCDL-Supplier-1]

Owner

Group: ***Security***

NCWF Role: ***Systems Security Analyst***

Administration

security	devops	development	risk
R	-	-	-

The Autonomous Vehicle Cybersecurity Manufacturer Disclosure Statement (AVCMDS) details the practices and capabilities of a supplier under consideration.

Training Provided

yes

Phase Requirement Dependencies

none

External Group Dependencies

none

AVCDL Products

[AVCDL-Supplier-1.1] ***Autonomous Vehicle Cybersecurity Manufacturer Disclosure Statement***

9.9.2 Supplier Self-reported Maturity [AVCDL-Supplier-2]

Owner

Group: ***Security***

NCWF Role: ***Systems Security Analyst***

Administration

security	devops	development	risk
R	-	-	-

A supplier self-report maturity assessment serves as a basis for establishing the maturity of the supplier's cybersecurity processes. The lifecycle put forth in this document is assumed to be the basis for comparison to ensure that all necessary activities are included.

Training Provided

yes

Phase Requirement Dependencies

none

External Group Dependencies

none

AVCDL Products

[AVCDL-Supplier-2.1] ***Supplier Self-reported Cybersecurity Maturity Assessment***

9.9.3 Cybersecurity Interface Agreement [AVCDL-Supplier-3]

Owner

Group: ***Security***

NCWF Role: ***Systems Security Analyst***

Administration

security	devops	development	risk
R	-	-	-

The Cybersecurity Interface Agreement (CIA) legally establishes the relationship between the supplier and customer.

Training Provided

yes

Phase Requirement Dependencies

[AVCDL-Foundation-4.2] ***Global Security Requirements***

[AVCDL-Supplier-1.1] ***Autonomous Vehicle Cybersecurity Manufacturer Disclosure Statement***

[AVCDL-Supplier-2.1] ***Supplier Self-reported Cybersecurity Maturity Assessment***

External Group Dependencies

none

AVCDL Products

[AVCDL-Supplier-3.1] ***Cybersecurity Interface Agreement***

10. Requirement Role Assignments

The following table shows the ***AVCDL*** process requirement role assignments.

Requirement	Name	Group	NCWF Title
Foundation-1	Training	***Security***	***Cyber Instructor***
Foundation-2	Roles and Responsibilities	***Security***	***Systems Requirements Planner***
Foundation-3	Toolchain Support	***Devops***	***Information Systems Security Developer***
Foundation-4	Definition of Security Requirements	***Security***	***Systems Requirements Planner***
Foundation-5	Protect the Code	***Devops***	***Information Systems Security Developer***
Foundation-6	Ensure Release Integrity	***Devops***	***Information Systems Security Developer***
Foundation-7	Incident Response Plan	***Security***	***Partner Integration Planner***
Foundation-8	Decommissioning Plan	***Security***	***Partner integration Planner***
Foundation-9	Threat Prioritization Plan	***Security***	***Systems Requirements Planner***
Foundation-10	Deployment Plan	***Devops***	***Information Systems Security Developer***
Requirements-1	Definition of Security Requirements	***Security***	***Security Architect***
Requirements-2	Requirements Gate	***Security***	***Secure Software Assessor***
Design-1	Take Security Requirements and Risk Information into Account During Software Design	***Development***	***Software Developer***
Design-2	Review the Software Design to Verify Compliance with Security Requirements and Risk Information	***Security***	***Systems Requirements Planner***
Design-3	Attack Surface Reduction	***Security***	***Security Architect***
Design-4	Threat Modeling	***Security***	***Security Architect***
Design-5	Design Gate	***Security***	***Secure Software Assessor***

Implementation-1	Use Approved Tools	Development	Software Developer
Implementation-2	Configure the Compilation and Build Process to Improve Executable Security	Devops	Information Systems Security Developer
Implementation-3	Configure the Software to Have Secure Settings by Default	Security	Security Architect
Implementation-4	Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality	Development	Software Developer
Implementation-5	Create Source Code Adhering to Secure Coding Practice	Development	Software Developer
Implementation-6	Deprecate Unsafe Functions	Development	Software Developer
Implementation-7	Static Analysis	Devops	Information Systems Security Developer
Implementation-8	Dynamic Program Analysis	Development	Software Developer
Implementation-9	Security Code Review	Security	Secure Software Assessor
Implementation-10	Fuzz Testing	Security	Vulnerability Assessment Analyst
Implementation-11	Implementation Gate	Security	Secure Software Assessor
Verification-1	Penetration Testing	Security	Vulnerability Assessment Analyst
Verification-2	Threat Model Review	Security	Security Architect
Verification-3	Attack Surface Analysis Review	Security	Security Architect
Verification-4	Verification Gate	Security	Secure Software Assessor
Release-1	Final Security Review	Security	Secure Software Assessor
Release-2	Archive	Devops	Information Systems Security Developer
Release-3	Release Gate	Security	Secure Software Assessor
Operation-1	Identify and Confirm Vulnerabilities on an Ongoing Basis	Security	Cyber Defense Incident Responder
Operation-2	Assess and Prioritize the Remediation of all Vulnerabilities	Security	Cyber Defense Forensics Analyst

Operation-3	Analyze Vulnerabilities to Identify Their Root Causes	***Security***	***Cyber Defense Forensics Analyst***
Operation-4	Secure Deployment	***Devops***	***Information Systems Security Developer***
Decommissioning-1	Decommissioning Protocol	***Devops***	***Information Systems Security Developer***
Supplier-1	AVCMDS	***Security***	***Systems Security Analyst***
Supplier-2	Supplier Self-reported Maturity	***Security***	***Systems Security Analyst***
Supplier-3	Cybersecurity Interface Agreement	***Security***	***Systems Security Analyst***

Table 2 - Requirement Role Assignments

11. Groups

This section summarizes the AVCDL phase requirements by group showing those requirements for which they have primary responsibility (*accountable* in RACI parlance).

The following is a summary mapping of the phase requirements to the groups involved: (highlight shows group accountable)

		Title	security	devops	development	risk
Foundation	1	Training	R	I	C	I
	2	Roles and Responsibilities	R	C	C	
	3	Toolchain Support	C	R	C	
	4	Definition of Security Requirements	R	I	I	
	5	Protect the Code	C	R		
	6	Ensure Release Integrity	C	R	C	
	7	Incident Response Plan	R		C	
	8	Decommissioning Plan	R	C	C	
	9	Threat Prioritization Plan	R		I	R
	10	Deployment Plan	C	R	C	
Requirements	1	Security Requirements Definition	R		I	
	2	Requirements Gate	R		R	
Design	1	Apply Security Requirements and Risk Information to Design	R		R	
	2	Security Design Review	R		R	C
	3	Attack Surface Reduction	R		R	
	4	Threat Modeling	R		R	
	5	Design Gate	R		R	R
Implementation	1	Use Approved Tools	C	C	R	
	2	Configure the Compilation and Build Process to Improve Executable Security	C	R	C	
	3	Configure the Software to Have Secure Settings by Default	R		R	
	4	Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality	C	I	R	
	5	Create Source Code Adhering to Secure Coding Practice	C		R	
	6	Deprecate Unsafe Functions	C		R	
	7	Static Analysis	C	R	C	
	8	Dynamic Program Analysis	C		R	
	9	Security Code Review	R		C	
	10	Implementation Gate	R	R	R	
	11	Fuzz Testing	R	C	C	
Verification	1	Penetration Testing	R	C	C	
	2	Threat Model Review	R		R	
	3	Attack Surface Analysis Review	R		R	
	4	Verification Gate	R		R	R
Release	1	Final Security Review	R	C	C	C
	2	Archive		R	C	
	3	Release Gate	R	R	R	R
Operation	1	Identify and Confirm Vulnerabilities on an Ongoing Basis	R		C	
	2	Assess and Prioritize the Remediation of all Vulnerabilities	R		C	C
	3	Analyze Vulnerabilities to Identify Their Root Causes	R		C	
	4	Secure Deployment	C	R	C	
Decommissioning	1	Apply Decommissioning Protocol	I	R		

Table 3 – AVCDL Phase Requirement - Group Mapping

Note: Information regarding RACI is at:
https://en.wikipedia.org/wiki/Responsibility_assignment_matrix

11.1 Groups [Devops]

The following process requirements are the responsibility of devops:

Requirement	Description
Foundation-3	Toolchain Support
Foundation-5	Protect the Code
Foundation-6	Ensure Release Integrity
Foundation-10	Deployment Plan
Implementation-2	Configure the Compilation and Build Process to Improve Executable Security
Implementation-7	Static Analysis
Release-2	Archive
Operation-4	Secure Deployment
Decommissioning-1	Decommissioning Protocol

Table 4 - Devops Requirement Responsibilities

11.2 Groups [Development]

The following process requirements are the responsibility of development:

Requirement	Description
Design-1	Take Security Requirements and Risk Information into Account During Software Design
Implementation-1	Use Approved Tools
Implementation-4	Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality
Implementation-5	Create Source Code Adhering to Secure Coding Practice
Implementation-6	Deprecate Unsafe Functions
Implementation-8	Dynamic Program Analysis

Table 5 - Development Requirement Responsibilities

11.3 Groups [Security]

The following process requirements are the responsibility of security:

Requirement	Description
Foundation-1	Training
Foundation-2	Roles and Responsibilities
Foundation-4	Definition of Security Requirements
Foundation-7	Incident Response Plan
Foundation-8	Decommissioning Plan
Foundation-9	Threat Prioritization Plan
Requirements-1	Definition of Security Requirements
Requirements-2	Requirements Gate
Design-2	Review the Software Design to Verify Compliance with Security Requirements and Risk Information
Design-3	Attack Surface Reduction
Design-4	Threat Modeling
Design-5	Design Gate
Implementation-3	Configure the Software to Have Secure Settings by Default
Implementation-10	Fuzz Testing
Implementation-11	Implementation Gate
Verification-1	Penetration Testing
Verification-2	Threat Model Review
Verification-3	Attack Surface Analysis Review
Verification-4	Verification Gate
Release-1	Final Security Review
Release-3	Release Gate
Operation-1	Identify and Confirm Vulnerabilities on an Ongoing Basis
Operation-2	Assess and Prioritize the Remediation of all Vulnerabilities
Operation-3	Analyze Vulnerabilities to Identify Their Root Causes
Supplier-1	AVCMDS
Supplier-2	Supplier Self-reported Maturity
Supplier-3	Cybersecurity Interface Agreement

Table 6 - Security Requirement Responsibilities

11.4 Groups [Risk]

Note: There are no process requirements for which the risk group has primary responsibility.

12. Reference Documents

This section contains a description of the reference documents relating to the ***AVCDL***.

12.1 Standards

1. **Cybersecurity Maturity Model Certification (CMMC)**
   https://www.acq.osd.mil/cmmc/docs/CMMC_ModelMain_V1.02_20200318.pdf
2. **Systems and software engineering - Software life cycle processes (ISO/IEC/IEEE 12207)**
   https://en.wikipedia.org/wiki/ISO/IEC_12207
3. **Systems and software engineering - System life cycle processes (ISO/IEC 15288)**
   https://en.wikipedia.org/wiki/ISO/IEC_15288
4. **Road Vehicles – Cybersecurity Engineering (ISO/SAE 21434)**
   https://www.sae.org/standards/content/iso/sae21434/
5. **Systems Security Engineering - Capability Maturity Model (SSE-CMM)**
   https://www.iso.org/standard/44716.html
6. **Microsoft Security Development Lifecycle (SDL) - simplified implementation**
   http://download.microsoft.com/download/F/7/D/F7D6B14F-0149-4FE8-A00F-0B9858404D85/Simplified_Implementation_of_the_SDL.doc
7. **NHTSA Cybersecurity Best Practices for the Safety of Modern Vehicles**
   https://www.nhtsa.gov/staticfiles/nvs/pdf/812333_CybersecurityForModernVehicles.pdf
8. **Guidelines for the Creation of Interoperable Software Identification (SWID) Tags**
   https://nvlpubs.nist.gov/nistpubs/ir/2016/NIST.IR.8060.pdf
9. **Protecting Controlled Unclassified Information in Nonfederal Systems and Organizations**
   https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-171r2.pdf
10. **NICE Cybersecurity Workforce Framework (NCWF)**
    https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-181r1.pdf
11. **Secure Software Development Framework (SSDF)**
    https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218-draft.pdf
12. **Static Analysis Results Interchange Format (SARIF)**
    https://docs.oasis-open.org/sarif/sarif/v2.0/csprd02/sarif-v2.0-csprd02.pdf
13. **Systems Engineering Capability Maturity Model (CMM)**
    https://resources.sei.cmu.edu/asset_files/TechnicalReport/1993_005_001_16211.pdf
14. **Software Package Data Exchange (SPDX®) Specification**
    https://spdx.dev/wp-content/uploads/sites/41/2020/08/SPDX-specification-2-2.pdf
15. **Proposal for a Recommendation on Cyber Security**
    https://unece.org/DAM/trans/doc/2019/wp29grva/ECE-TRANS-WP29-GRVA-2019-02e.pdf

16. **EVITA D2.3 Security requirements for automotive on-board networks based on dark-side scenarios**
https://zenodo.org/record/1188418/files/EVITAD2.3v1.1.pdf?download=1
17. **Road Vehicles – Functional Safety (ISO 26262)**
https://www.iso.org/publication/PUB200262.html
18. **Road Vehicles – Software Update Engineering (ISO 24089)**
https://www.iso.org/standard/77796.html
19. **UN Regulation No. 155 - Cyber security and cyber security management system**
https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security
20. **UN Regulation No. 156 - Software update and software update management system**
https://unece.org/transport/documents/2021/03/standards/un-regulation-no-156-software-update-and-software-update

12.2 Secondary Documents

All secondary documents as named below are located relative to this file in the following folder:

```
./reference_documents/secondary_documents
```

General Information

AVCDL Phase Requirement Product ISO/SAE 21434 Work Product Fulfillment Summary *[PDF]*

AVCDL Phase Requirement Product ISO 26262 Work Product Fulfillment Summary *[PDF]*

Ranked / Risked Threat Report ..... *[PDF]*

Threat Report ..... *[PDF]*

Understanding the Phase Product Dependencies Graph ..... *[PDF]*

Understanding Workflow Graphs ..... *[PDF]*

Foundation Phase

Code Protection Plan ..... *[PDF]*

Cybersecurity Monitoring Plan ..... *[PDF]*

Cybersecurity Requirements Catalog ..... *[PDF]*

Decommissioning Plan ..... *[PDF]*

Deployment Plan ..... *[PDF]*

Global Security Goals ..... *[PDF]*

Global Security Requirements ..... *[PDF]*

Incident Response Plan ..... *[PDF]*

List of Approved Tools and Components ..... *[PDF]*

Release Integrity Plan ..... *[PDF]*

Security Requirements Taxonomy ..... *[PDF]*

System to Track Training Participation ..... *[PDF]*

Threat Prioritization Plan ..... *[PDF]*

Training Catalog ..... *[PDF]*

Requirements Phase

Element-level Security Goals ..... *[PDF]*

Element-level Security Requirements ..... *[PDF]*

Requirements Phase Gate ..... *[PDF]*

Design Phase

Attack Surface Analysis Report ..... ***[PDF]***

Design Phase Gate ..... ***[PDF]***

Design Showing Security Considerations ..... ***[PDF]***

Security Design Review Report ..... ***[PDF]***

Threat Modeling Report ..... ***[PDF]***

Implementation Phase

Build Process Documentation ..... ***[PDF]***

Component / Version - Product / Version Cross-reference Document ..... ***[PDF]***

Currently Used Deprecated Functions Document ..... ***[PDF]***

Dynamic Analysis Report ..... ***[PDF]***

Fuzz Testing Report ..... ***[PDF]***

Implementation Phase Gate ..... ***[PDF]***

List of Tools and Components Used ..... ***[PDF]***

Secure Code Review Summary ..... ***[PDF]***

Secure Development ..... ***[PDF]***

Secure Settings Document ..... ***[PDF]***

Static Analysis Report ..... ***[PDF]***

Verification Phase

Penetration Testing Report ..... ***[PDF]***

Updated Attack Surface Analysis ..... ***[PDF]***

Updated Threat Model ..... ***[PDF]***

Verification Phase Gate ..... ***[PDF]***

Release Phase

Archive Manifest ..... ***[PDF]***

Final Security Review Report ..... ***[PDF]***

Release Phase Gate ..... ***[PDF]***

Operation Phase

Cybersecurity Incident Report ..... ***[PDF]***

Software Deployment Report ..... ***[PDF]***

Decommissioning Phase

Decommissioning Report ..... ***[PDF]***

Supplier Processes

Autonomous Vehicle Cybersecurity Manufacturer Disclosure Statement ..... ***[PDF]***

Supplier Self-reported Maturity Assessment ..... ***[PDF]***

Cybersecurity Interface Agreement ..... ***[PDF]***

12.3 Working Material

All working material spreadsheets as named below are located relative to this file in the following folder:

`./reference_documents/working_material`

AVCDL CMMC ..... ***[XSLX]***

AVCDL mappings ..... ***[XSLX]***

AVCDL roles and responsibilities ..... ***[XSLX]***

12.4 Templates

All templates as named below are located relative to this file in the following folder:

`./reference_documents/templates`

AVCDL CMM progress template ..... ***[XSLX]***

AVCDL cybersecurity interface agreement summary template ..... ***[XSLX]***

AVCDL Cybersecurity Interface Agreement template ..... ***[DOCX]***

AVCDL vendor CMM template ..... ***[XSLX]***

AVCMDS Worksheet template ..... ***[XSLX]***

vendor process - AVCDL product mapping template ..... ***[XSLX]***

13. Continuous Improvement Progress Summary Example

The following table shows a possible representation for tracking the maturity of the ***AVCDL*** implementation:

AVCDL phase	requirement	name	implementation	CMM level
Foundation	1	Training	partially implemented	1
	2	Roles and Responsibilities	partially implemented	1
	3	Toolchain Support	partially implemented	1
	4	Definition of Security Requirements	partially implemented	1
	5	Protect the Code	implemented	2
	6	Ensure Release Integrity	not implemented	0
	7	Incident Response Plan	partially implemented	1
	8	Decommissioning Plan	not implemented	0
	9	Threat Prioritization Plan	not implemented	0
	10	Deployment Plan	not implemented	0
Requi	1	Definition of Security Requirements	partially implemented	1
	2	Requirements Gate	partially implemented	1
Design	1	Consider Security and Risk During Design	partially implemented	1
	2	Design Review	partially implemented	1
	3	Attack Surface Reduction	not implemented	0
	4	Threat Modeling	partially implemented	1
	5	Design Gate	not implemented	0
Implementation	1	Use Approved Tools	implemented	2
	2	Configure Process for Security	partially implemented	1
	3	Configure Secure Settings by Default	partially implemented	1
	4	Reuse Well-Secured Software	implemented	2
	5	Use Secure Coding Practice	partially implemented	1
	6	Deprecate Unsafe Functions	partially implemented	1
	7	Static Analysis	implemented	2
	8	Dynamic Program Analysis	partially implemented	1
	9	Security Code Review	partially implemented	1
	10	Fuzz Testing	partially implemented	1
	11	Implementation Gate	not implemented	0
Verification	1	Penetration Testing	implemented	1
	2	Threat Model Review	not implemented	0
	3	Attack Surface Analysis Review	not implemented	0
	4	Verification Gate	not implemented	0
Release	1	Final Security Review	not implemented	0
	2	Archive	partially implemented	1
	3	Release Gate	not implemented	0
Operation	1	Identify and Confirm Vulnerabilities	partially implemented	1
	2	Assess and Prioritize the Remediation	not implemented	0
	3	Root Cause Vulnerabilities	partially implemented	1
	4	Secure Deployment	not implemented	0
De	1	Apply Decommissioning Protocol	not implemented	0

Table 7 - Maturity Tracking Example